

BaW OS: Investigación de Diseño para una Plataforma Agent-Native de Operaciones Inmobiliarias

Síntesis de cuatro investigaciones primarias sobre UX agent-native, command centers, colaboración humano-agente y tendencias agentic SaaS 2025–2026, aplicada al diseño de BaW OS.

Resumen Ejecutivo

El diseño de software empresarial está atravesando el cambio de paradigma más consecuente de los últimos 60 años: la interfaz ha dejado de ser el lugar donde se toman las decisiones para convertirse en una superficie de supervisión sobre un sistema operado por agentes (UX Tigers, AI Space). Jakob Nielsen lo describe como el paso de "operador" a "supervisor": el usuario ya no conduce un coche, gestiona un chofer.

Para una plataforma de operaciones inmobiliarias como BaW OS, esta transición no es opcional. La unidad operativa —el edificio, con sus unidades, contratos, reservas, mantenimientos, cobros y experiencia de huésped o inquilino— es exactamente el tipo de sistema donde los agentes agregan más valor: alto volumen de decisiones repetitivas con excepciones críticas, múltiples stakeholders (desarrolladores, owners, ops, conserjes, inquilinos) con niveles distintos de delegación y una fuerte presión por operar 24/7 sin incrementar plantilla proporcionalmente.

Tres hallazgos centrales articulan la tesis del producto:

1. **Humans supervise, agents operate.** Los agentes monitorean 4,847 items; el humano revisa los 3 que requieren atención (LBZ Advisory). El "inbox zero" agéntico debe diseñarse como un estado operativo de primera clase, no como ausencia de alertas.
2. **Confianza como output del diseño, no de la tecnología.** Smashing Magazine lo sintetiza: "Autonomy is an output of a technical system. Trustworthiness is an output of a design process" (Smashing Magazine). La transparencia, el intent preview, el explainable rationale, el action receipt y la autonomy dial son los componentes UX que convierten capacidad técnica en confianza operativa.
3. **La interfaz como control plane.** El trabajo real ocurre en la capa de agentes; la UI existe para configurar políticas, escalar excepciones, auditar y tomar control. Esto convierte a BaW OS menos en un "CRM de edificios" y más en un sistema operativo de edificios con una capa de mission control sobre él (AI Space).

La tesis central, en una línea: *BaW OS es el sistema operativo de un edificio; los humanos definen política y atienden excepciones; los agentes ejecutan operaciones continuas 24/7 bajo auditoría y delegación trazable.*

Parte 1: El Cambio de Paradigma — De SaaS Tradicional a Software Agent-Native

1.1 El giro de operador a supervisor

Nielsen articula la transformación así: *"In command-based interaction, you strike every blow (click every icon) to gradually produce what you want... Users are changing from doing the work (operating the UI) to supervising the work. The correct analogy is no longer driving a car; it is managing a chauffeur"* ([UX Tigers](#)).

Esto obliga a la UX a evolucionar en cinco dimensiones ([UX Tigers](#)):

- 1. **Modelo de interacción:** basado en comandos → basado en intención
- 2. **Métricas de usabilidad:** tiempo de tarea → calidad del outcome y confianza de delegación
- 3. **Capas de orquestación:** un único usuario → múltiples stakeholders + sub-agentes
- 4. **Fricción calibrada:** no toda fricción es mala; los "speed bumps" antes de acciones irreversibles son features
- 5. **Interacción exploratoria:** clarificar intención mediante descubrimiento, no mediante formularios

1.2 El modelo de triple capa (Nielsen)

Los sistemas intent-based maduros se están asentando en una arquitectura de tres capas ([UX Tigers](#)):

Capa	Función	Implicación para BaW OS
Intent Surface	Usuario declara el outcome; multimodal; idealmente infiere intención del contexto	Comandos naturales como "cobra los atrasos del edificio Vallarta" o "prepara la unidad 204 para el check-in de mañana"
Orchestration Surface	Capa donde se negocia entre intención y ejecución: plan, consent, receipts, provenance, intents conflictivos	La más crítica y la que más equipos ignoran. En BaW OS: el "approval hub" y los flujos de excepción
Direct Manipulation Surface	La GUI tradicional persiste como fallback: edge-cases, correcciones, override. Pero ahora se manipulan planes, no controles crudos	Las vistas clásicas tipo tabla y formulario se mantienen para auditoría, corrección y overrides, pero dejan de ser el primary surface

1.3 Qué muere en el SaaS tradicional

Seis patrones están siendo reemplazados ([Oliv](#), [a16z](#), [Flashdba](#)):

Patrón tradicional	Sustituido por
Dashboard estático multi-widget	Interfaz exception-first + agent inbox
Formulario de entrada manual	Agentes que leen intención y escriben datos; humanos confirman

Patrón tradicional	Sustituido por
Campana de notificaciones con volumen indiferenciado	Oversight inbox calibrado por tipo de acción requerida
Búsqueda basada en keywords	Síntesis LLM: "¿por qué cayó la ocupación?"
Pantallas CRUD	Agentes escriben en el system of record; humanos revisan
Automatizaciones rígidas tipo IFTTT	Agentes que perciben contexto y negocian excepciones

Crucial: Satya Nadella ha anticipado que *"business applications as we know them will collapse in the agent era. They are essentially CRUD databases with business logic that will migrate into the AI tier"* ([Glean Perspectives](#)).

1.4 La interfaz como control plane

"The interface used to be where decisions were made. Now it is a monitoring surface. A configuration panel. A place for oversight. The real work happens in the agent layer sitting above it" ([AI Space](#)).

La interfaz de BaW OS cumple cuatro funciones irreducibles:

- Monitorear comportamiento del agente y cazar errores antes de que escalen
- Revisar outputs antes de que lleguen a owners, inquilinos o sistemas externos
- Definir reglas, permisos y límites para lo que los agentes pueden hacer
- Manejar excepciones fuera del scope decisional del agente

Sin embargo, los sistemas de registro (system of record) ganan importancia, no la pierden: *"The more AI Agents work, the more important this stable core becomes"* ([Mercury Media Technology](#)). El edificio, la unidad, el contrato, el cobro son objetos de primera clase cuya integridad debe protegerse con identidad de agente trazable.

Parte 2: Patrones UX Agent-Native Fundamentales

2.1 Los seis patrones del lifecycle (framework Smashing Magazine)

Publicado por [Smashing Magazine](#), es el framework práctico más citado. Estructura la UX alrededor del ciclo de vida del agente:

Pre-acción (establecer intención):

1. **Intent Preview (Plan Summary):** Resumen en lenguaje claro, pasos secuenciales, tres affordances [Proceed] [Edit Plan] [Handle it Myself]. Meta: >85% de planes aceptados sin editar; si Handle it Myself supera el 10%, el modelo necesita revisión.
2. **Autonomy Dial:** Configuración per-workflow que calibra la independencia del agente. Cuatro niveles: *Observe & Suggest* → *Plan & Propose* → *Act with Confirmation* → *Act Autonomously*. La autonomía se gana, no se regala ([TianPan](#)).

En-acción (proveer contexto):

- 3. **Explainable Rationale ("Why?" Pattern):** Responde proactivamente "¿Por qué el agente hizo eso?" con la fórmula *"Because you said X, I did Y"*. Anclado en preferencias declaradas del usuario.
- 4. **Confidence Signal:** El agente surfacea su nivel de confianza para prevenir sesgo de automatización. Score numérico + scope declaration + cues visuales (check verde alta, signo interrogación amarillo baja).

Post-acción (seguridad y recuperación):

- 5. **Action Audit & Undo:** Timeline cronológico con estados claros (Successful / In Progress / Undone), ventanas de undo explícitas ("Undo available for 15 minutes") y comunicación honesta cuando algo ya no es reversible.
- 6. **Escalation Pathway:** La escalación es un feature de primera clase, no un estado de error. Tres tipos: clarificación, presentación de opciones, petición de takeover humano.

2.2 La librería de 12 patrones (HatchWorks)

HatchWorks AI propone una taxonomía de 12 patrones prácticos con orden de implementación (controls → receipts → logs → approvals → memory → eval):

#	Patrón	Reemplaza a	Elemento central
1	Taskboard + Outcomes	Progreso solo-chat	Goals → tasks con owner (agent/human), estado, SLA
2	Activity Timeline	Pasos misteriosos	Log cronológico filtrable por severidad
3	Start/Stop/Pause/Resume	Ejecución implícita	Controles async explícitos con semántica clara
4	Autonomy Levels	Automatización binaria	Slider Suggest → Draft → Execute
5	Two-Phase Actions	Ejecución silenciosa	Plan → Validate → Execute con receipt
6	Action Receipts	Mensaje "Done!"	Qué cambió, dónde, por quién, timestamp, rollback
7	Evidence Panel	Chain-of-thought crudo	Fuentes curadas + rationale desafiables
8	Human Checkpoint Gates	HITL ad-hoc	Tipos definidos: plan/ejecución/output/excepción
9	Role Cards	Blob de prompt único	Rol, scope, tools, permisos por agente
10	Safe Failure & Recovery	Fallo silencioso	"Safe mode" determinista + estado "I'm stuck"
11	Memory Controls	Memoria oculta	"Qué recuerdo / por qué / editar"
12	Budget + Time Boxes	Ejecución ilimitada	Max spend + max time con ETA en vivo

2.3 El framework ADEPTS (académico)

ADEPTS ([arXiv 2507.15885](#)) define seis capacidades user-facing para agentes y funciona como vocabulario compartido entre diseño, ingeniería y política:

- Awareness: el agente sabe lo que está pasando
- Delegation: maneja tareas en nombre del usuario
- Explainability: articula su razonamiento
- Persistence: mantiene continuidad entre sesiones
- Transparency: muestra su estado y sus limitaciones
- Steering: los usuarios pueden guiar, corregir y redirigir

2.4 Visibilidad del agente: cuatro dimensiones

Onething Design articula cuatro dimensiones que deben estar diseñadas explícitamente:

1. **Intent Visibility:** ¿qué intenta lograr el agente? El usuario ve el goal, no solo acciones.
2. **Process Visibility:** ¿qué pasos ha dado, en qué orden y por qué?
3. **State Visibility:** ¿qué está haciendo ahora? ¿running, waiting, blocked, done?
4. **Outcome Visibility:** ¿qué cambió en el mundo como resultado?

Microsoft's Agent UX Principles lo articulan de forma prescriptiva: *"Agent status, or what the agent is doing, is clearly visible at all times"* ([Microsoft Design](#)).

2.5 Action queues y arquitectura de aprobación

El principio canónico ([Stack AI](#)): separar intent de execution. El agente *propone* guardando un payload estructurado en almacenamiento durable; solo con aprobación explícita *commit-ee* la acción con keys de idempotencia.

La clasificación de tres niveles ([TianPan](#)):

AUTO	→ Ejecuta sin aprobación (reversible, bajo riesgo, blast radius pequeño)
LOG	→ Ejecuta y registra para revisión async (spot-check en schedule)
REQUIRE_APPROVAL	→ Bloquea hasta aprobación (irreversible, financiero, bulk outbound)

El Evidence Pack (lo que el revisor necesita para decidir en 10–30 segundos):

- Qué acción se propone (específica, con parámetros)
- Por qué el agente cree que es correcta (reasoning breve)
- Qué pasa si se aprueba vs. rechaza (consecuencias downstream)
- Ventana de tiempo antes de timeout o escalamiento
- Data soporte: registros afectados, historial, risk score

Acciones más allá de approve/reject: approve with edits, delegate, snooze/defer, request more context, reject with reason.

2.6 Exception handling y escalación

Cinco patrones para excepciones agentic ([UXmatters](#), [Microsoft Research](#)):

1. **Escalación como feature:** "Emergency stops should exist not as hidden safeguards, but as visible assurances."
2. **Tres tipos:** clarification request, options presentation, human takeover request.
3. **After-rejection behavior explícita:** Abort / Reroute / Escalate — nunca reintentar silenciosamente una acción rechazada.
4. **Safe mode fallback:** el agente transita a pasos deterministas cuando la incertidumbre sube; el usuario ve la transición.
5. **Estado "I'm stuck":** el agente declara que no puede proceder y entrega un plan resumible al humano.

Anthropic observó algo contraintuitivo: los agentes se auto-detienen más de lo que los humanos los interrumpen. Top razones (500K sesiones de Claude Code): presentar opciones entre aproximaciones (35%), gather diagnóstico (21%), clarificar (13%), credenciales faltantes (12%), aprobación antes de acción consecuente (11%) ([Anthropic](#)).

2.7 Audit trail para sistemas humano+agente

La pregunta cambia: de *"¿qué hizo el usuario?"* a *"¿qué hizo el agente, qué aprobó el humano, y podría alguien reconstruir por qué cada decisión se tomó?"* ([MarkTechPost](#)).

Cinco requisitos de auditoría empresarial ([Seekr](#)):

Requisito	Significado
Training data attribution	Trazar outputs a la data que los moldeó
Influence scoring	Ranear contribuciones por nivel de impacto
Complete audit trails	Cada decisión, input, output, reasoning step con timestamp
Contestability	Los humanos pueden desafiar, trazar y corregir outputs
Human actions as first-class events	Intervenciones humanas logueadas en el mismo sistema que acciones del agente

Patrón de audit log unificado (actor visualmente distinguido):

```
[AGENT] 12:03:42 Identified 47 overdue rent invoices
[AGENT] 12:03:44 Generated dunning batch draft
[HUMAN] 12:05:12 María García approved $143,200 batch (reviewed 90s)
[AGENT] 12:05:13 Submitted notifications via SMS/Email
[SYSTEM] 12:05:18 Checkpoint: rollback window expires 12:35:18
```

Mínimos de atribuibilidad por acción (Augment Code): Agent ID/role/version, model name/version, prompt template hash, input data commitment hash, confidence score, delegation contract reference, human review status.

Para compliance con EU AI Act (agosto 2026, hasta €35M en sanciones): logging como requisito arquitectónico, no procedural — Article 12(1) exige que los sistemas de alto riesgo "technically allow" registro automático de eventos.

2.8 Progressive disclosure del razonamiento

Modelo de tres niveles (Smashing Magazine, The Interactive Studio):

- Nivel 1 — What (visible por defecto): "Agent completó: borrador de email a inquilino por disputa de cobro"
- Nivel 2 — Why (expandible): 2-3 factores anclados en preferencias del usuario
- Nivel 3 — How (power users / auditores): secuencia completa de tool calls, data intermedia, árbol de decisión

Principios: nunca exponer API calls o prompt crudo al usuario final; anclar en preferencias declaradas; el nivel 3 es para compliance y debugging, no uso diario.

2.9 La máquina de estados completa del trabajo agéntico

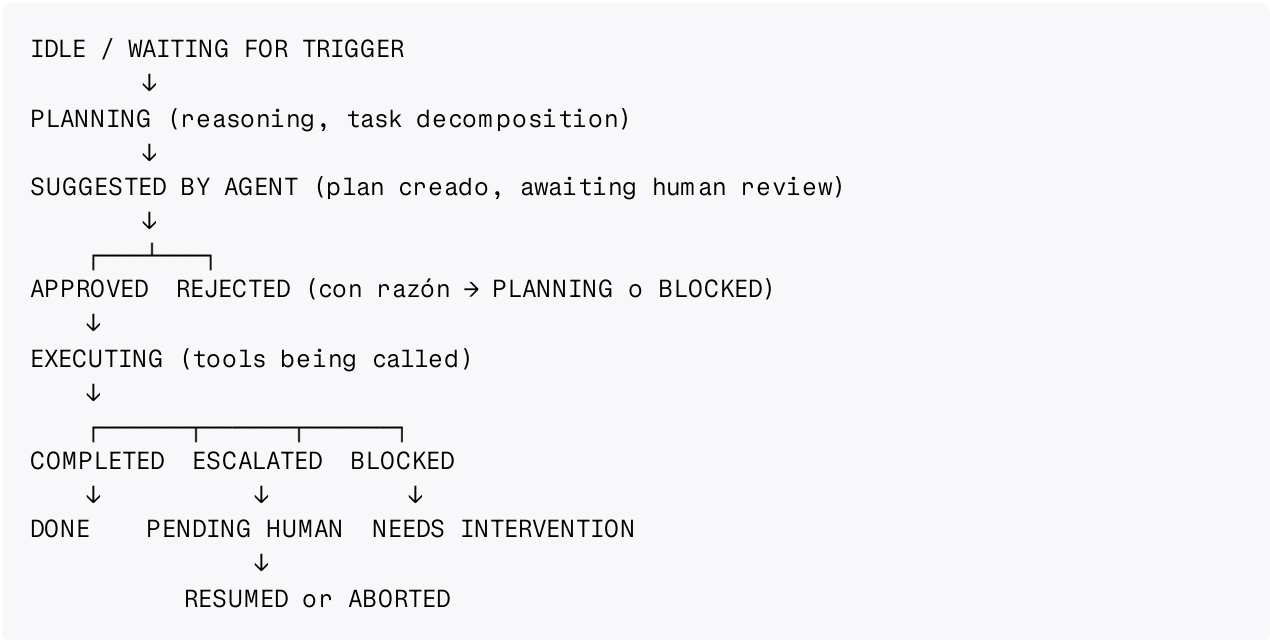


Tabla de tratamiento visual por estado (aplicable directamente a BaW OS):

Estado	Tratamiento visual	Acción requerida	Data a mostrar
Suggested by Agent	Badge ámbar / pill "Pending Review"	Review + approve/edit/reject	Intent Preview, evidence pack, confidence
Pending Approval	Icono reloj, countdown SLA	Approve / reject / delegate	Asignado a, tiempo restante para escalamiento

Estado	Tratamiento visual	Acción requerida	Data a mostrar
Approved	Badge azul	Ninguna (agente ejecuta)	Timestamp aprobación, nombre approver
Executing	Progress animado	Pause / stop disponibles	Paso actual, % complete, ETA, costo consumido
Completed	Check verde	Opcional: review receipt	Action receipt: qué cambió, diffs, rollback
Executed by Agent (autonomous)	Badge verde + tag "By Agent"	Opcional: undo en ventana	Igual + "Undo disponible por X min"
Escalated to Human	Badge naranja + assignee	Humano debe actuar	Contexto completo + lo que el agente intentó
Blocked	Badge rojo	Intervención humana	Qué bloquea, resolución sugerida, impacto
Failed / Needs Intervention	Badge rojo	Investigar + decidir	Error, partial state, rollback, recovery path
Undone	Tachado + tag "Reversed"	Ninguna	Estado before/after, quién inició, timestamp

Parte 3: Modelos de Confianza, Delegación y Autonomía

3.1 Arquitectura de confianza: dos clusters de usuarios

Research identifica dos clusters de usuarios con estrategias de confianza diferentes (Taylor & Francis):

- **Cluster 1** (alta ansiedad IA): prefiere explicabilidad y certificación externa; busca múltiples señales simultáneas
- **Cluster 2** (pragmático): confía por desempeño observado; negativo hacia explicabilidad excesiva; valora reliability y fairness metrics

Implicación: BaW OS no debería construir una sola interfaz de confianza. Algunos usuarios (owners conservadores, CFOs) querrán entender el razonamiento; otros (ops managers experimentados) querrán evidencia de outcomes. Ninguno está equivocado.

El framework Trustworthy AI de Deloitte identifica cuatro dimensiones (Deloitte): reliability, capability, transparency, humanity. Dato clave: los trabajadores que confían en sus agentes son 10x más propensos a verlos como críticos para crear valor.

3.2 El framework de cinco niveles (Knight Columbia)

El framework más riguroso define niveles por el *rol del usuario*, no por capacidad del agente (Knight Columbia):

Nivel	Rol del usuario	Comportamiento del agente	Modo de interacción
L1	Operator	Soporte on-demand; usuario conduce planning	Sugiere pero no ejecuta sin aprobación
L2	Collaborator	Planean y ejecutan en paralelo; back-and-forth frecuente	Workspace compartido; takeover en cualquier paso
L3	Consultant	Agente toma iniciativa sobre horizontes extensos	Consulta en momentos clave; pause/redirect por mensajes
L4	Approver	Agente autónomo; pausa solo en bloqueos	Usuario preespecifica qué requiere aprobación
L5	Observer	Autonomía total; solo emergency off-switch	Logs only

Insight clave: autonomía es una decisión de diseño independiente de la capacidad. Un agente muy capaz puede constreñirse a L2 donde L5 sería inapropiado.

3.3 El framework A0–A4 empresarial (CEI)

Taxonomía operativa complementaria para deployment ([CEI America](#)):

Nivel	Nombre	Qué puede hacer	Controles requeridos
A0	Advisory Only	Leer, resumir, redactar — sin system actions	Content checks, logging
A1	Assistive Execution	Actuar con aprobación humana	Approval gates, change logging
A2	Constrained Autonomy	Actuar dentro de scope estricto	Whitelisted tools, data scope
A3	Conditional Autonomy	Actuar según políticas + risk scores	Policy engine, anomaly detection, kill-switch
A4	Mission Autonomy (raro)	Casi autonomía total	Independent review board

3.4 Thresholds de confianza como controles dinámicos

Los thresholds funcionan como el boundary entre procesamiento automatizado y revisión humana ([LlamalIndex](#)):

Defaults iniciales: 95% para clasificación, 90% para generación ([Innovate 24-7](#)).

Marco por tier de riesgo ([Turing](#)):

- Tier 1: bajo riesgo, alto volumen, alta accuracy → autonomía total
- Tier 2: medio riesgo → HOTL con thresholds ajustados
- Tier 3: alto riesgo → HOTL permanente con SLAs
- Tier 4: regulado/irreversible → HITL con logging reforzado

Los thresholds deben ser **dinámicos**: tight durante cierre mensual, relax en volumen rutinario; si revisores aprueban >95% sin cambios, bajar threshold 2–3 puntos.

3.5 HITL vs. HOTL vs. autonomía total

Modelo	Quién revisa	Timing	Ahorros capturados
HITL	Cada output	Antes de acción	20–40%
HOTL	Solo excepciones (bajo threshold)	Solo en fallos	60–80%
Full autonomy	Nadie	Nunca (solo monitoring)	~100%

Hallazgo no-obvio de Anthropic sobre oversight: *"Oversight strategy more reliably shaped users' exposure to problematic actions than their ability to correct them"*. Plan-based strategies (aprobar el plan en vez de cada paso) redujeron acciones problemáticas con OR=0.25, p<0.001 ([arXiv oversight strategies](#)). Máximos checkpoints ≠ máxima seguridad.

3.6 Modelos de permisos y delegación

El modelo correcto es delegated access (Oso):

```
User Request → Agent → Identity Provider → Scoped Token (user permissions) → Downstream
```

El agente hereda la *identidad del usuario*, no una cuenta de servicio compartida. Cuando el usuario pierde acceso, el agente inmediatamente también.

Antipatrón: acceso wide via service account. Viola least-privilege y significa que un agente comprometido tiene acceso a todo.

Controles avanzados:

- **Just-in-time credentials**: el agente pide credenciales en runtime; se descartan después
- **Temporal elevation**: acceso elevado para eventos time-bounded con revocación automática
- **Behavioral authorization**: si un agente empieza bulk-download a las 3 AM, acceso se restringe automáticamente
- **Context-aware restrictions**: device trust, geolocation, hora del día

3.7 Override y take-control

Patrones de recuperación de control ([arXiv oversight](#)):

Estrategia	Mecanismo	Hallazgo
Action Confirmation	Approve por paso	Max agencia humana; alto overhead
Risk-Gated	Agente corre libre; escala en momentos consecuentes	Menor exposición a problemáticas

Estrategia	Mecanismo	Hallazgo
Supervisory Co-Execution	Humanogate a nivel de plan; agente ejecuta autónomo	Mayor reducción de acciones problemáticas
Structurally Enriched	Combina plan-level + step-level + risk-aware	Mejor ajuste subjetivo en alto riesgo

Mecanismos UX: emergency stop siempre visible (sin confirmación para el stop); pause-and-redirect; undo/rollback; take-over step; selective re-approval.

Lección de Tesla FSD: transferencias de control requieren briefing de contexto ([Stanford](#)).
Mostrarle al humano qué hacía el agente, qué cambió, qué incertidumbre lleva — antes de que tome control.

Parte 4: Coordinación Multi-Agente y Visibilidad

4.1 Patrones de arquitectura multi-agente

Cinco patrones core, cada uno con implicaciones UI distintas ([AgileSoftLabs](#), [Azure](#)):

Patrón	Estructura	Reto UI
Supervisor/Worker	Orchestrator + workers especializados	Mostrar decisiones del orchestrator, debug de routing
Peer-to-Peer	Agentes negocian como iguales	Visualizar trabajo concurrente, outcomes de negociación
Hierarchical	Management multi-nivel	Reflejar jerarquía organizacional
Pipeline/Sequential	Etapas ordenadas	Tracking de progreso, data transformándose
Marketplace/Auction	Agentes bid por tareas	Criterios transparentes de allocation

Para BaW OS, el patrón supervisor/worker es el default razonable: un orchestrator central rutea hacia agentes especializados (cobros, mantenimiento, comms con huéspedes, revenue management).

4.2 Agent roster / fleet management

Componentes clave:

- **Agent status panel:** availability, tarea actual, health/performance (análogo a dashboards de team humano)
- **Task pipeline view:** task IDs únicos, prioridad (LOW/MED/HIGH), estado (PENDING/ACTIVE/ASSIGNED/ABORTED/SUCCEEDED), agente asignado, progreso
- **Unique identity:** fotos/nombres/roles para agentes — Relevance AI permite custom images; social model reduce fricción de atribución
- **Capability registry:** qué puede hacer cada agente, qué tools tiene, nivel de autonomía

Dust.tt (Dust) articula el principio: "one agent, one job done well". Equipos que intentan un mega-agente terminan con agentes que no hacen nada bien.

4.3 Resolución de conflictos entre agentes

Tres modelos de autoridad (Arion Research):

1. **Hierarchical control**: supervisor (humano o agente) decide. Resolución rápida, bottleneck a escala.
2. **Federated**: autoridad específica por dominio. Escala mejor, requiere boundaries claros.
3. **Decentralized consensus**: peer-to-peer negotiation. Resiliente, lento para urgencias.

Priority rules cuando la autoridad es igual:

- "Safety over speed" — security agent override a performance optimizer
- "Customer satisfaction over cost reduction" — support agent veta medidas de cost-cutting

4.4 Performance monitoring

Approach de producción Relevance AI (Relevance AI):

- **Automatic evaluation**: LLM judges evalúan continuamente en criteria custom
- **Production performance tracking**: lifecycle build → test → monitor, análogo a software
- **Real-time alerts**: notificación instantánea al completar tasks o necesitar atención
- **Workforce Trigger Controls**: pause/resume/stop de workforces enteros rápidamente

Parte 5: Command Center y Diseño de Operaciones

5.1 Patrones de mission control

NASA Open MCT (UX Planet):

- "Object-oriented, user-composable" displays
- Árbol de objetos a la izquierda, visualización principal configurable, timeline en bottom
- Composición: telemetría, procedures, trayectoria con comportamientos consistentes
- Principio: "All controls and key information accessible without hunting"
- Upgrades añadieron color-coding (rojo = off-nominal) para reducir clutter

NOC (Network Operations Center):

- Filas de desks frente a video wall central
- Dual monitor: live monitoring + incident ticketing
- Alert hierarchy: Critical → page inmediato; High → team; Medium → tickets; Low → dashboard

Military command: redundancy y scalability desde el diseño inicial; workstations ajustables; seamless integration de AV components.

SCADA / HMI (ISA-101): paletas neutras con emphasis selectivo; layouts consistentes reducen cognitive load; role-based access restringe acciones.

5.2 Diseño de información densa (Bloomberg)

La Terminal Bloomberg es el gold standard de densidad ([Matt Ström-Awn](#)):

- Una pantalla: sparklines de índices + tablas con docenas de rows/columns + news scrolling + keyboard shortcuts
- "Temporal density is the real superpower": data carga instantáneamente; usuarios navegan docenas de charts en milisegundos
- Launchpad: workspace componible, user-customizable, componentes magnéticamente dockables

Tres tipos de densidad:

Tipo	Definición	Palanca
Visual	Cuántas cosas vemos en un espacio	Whitespace, grouping, tamaño
Información	Data-ink ratio (Tufte)	Remover non-data-ink
Temporal	Logrado por unidad de tiempo	Loading speed, response time

Fórmula unificadora: $UI\ density = \text{valor user} / (\text{tiempo} \times \text{espacio})$.

5.3 Exception-first design

Principios ([ProSense](#), [GlitchLabs](#)):

- Mostrar solo data que requiere atención; exception-based $\neq$ ocultar data, es revelarla thoughtfully
- Thresholds claros; color-code (Red=critical, Amber=warning, Green=normal)
- Alertas prominentes al top; minimizar data normal

Layout recomendado:

CRITICAL EXCEPTIONS & ALERTS	← top
TREND CHARTS (key metrics)	← middle
SUMMARY STATS + LINKS	← bottom

Task-centric vs. entity-centric: el error común es organizar pantallas por tablas (Users, Orders, Products). Los operadores piensan en tareas: "procesar reembolsos", "aprobar

verificaciones pendientes", "investigar flagged transactions". Cada vista debe responder: *¿qué requiere mi atención? ¿qué acción debo tomar? ¿qué contexto necesito?*

5.4 Split view / primary-detail

Patrón workhorse de operaciones ([Cloudscape](#), [PatternFly](#)):

PRIMARY (Table / Map / List)	DETAIL PANE (selected item props, metrics, actions)
------------------------------------	--

Side panel cuando ≤5 columns importantes; bottom panel cuando >5.

5.5 La arquitectura "Building Operating System" de 5 capas

Síntesis de [Dribbble BMS](#) + [LinkedIn Muzicka](#):

Capa	Función	Patrón UI
Command Layer	Overview system-wide, alertas, KPIs	Exception-first dashboard, RAG status
Operations Layer	Task management, team coord, scheduling	Readiness dashboard estilo Breezeway + GPS map
Occupant Layer	Experiencia guest/resident, comms	Unified inbox, automated messaging
Financial Layer	Revenue, ocupación, costos, reporting owner	Estilo Stripe financial operations
Analytics Layer	Trends, benchmarking, predictivo	Progressive disclosure summary → drill-down

El edificio se trata como un **objeto ontológico** (à la Palantir) — con sub-objetos (unidades, sistemas, reservas, tareas, staff) con estado consistente, componibles en vistas por rol.

5.6 Table-centric vs. card-centric

Patrón	Ideal para	Características
Table	Comparar muchos items, scan, sort/filter	Alta densidad, column alignment, tabular nums
Card	Browsing visual, nuevos usuarios	Menor densidad, onboarding fácil, más scroll
List	Secuencial, status simple	Entre table y card; bueno para action-centric
Kanban	Workflow stages, drag-and-drop	Claridad de workflow, poco detalle por card

Ops teams generalmente prefieren table-centric para monitoring y triage. Cards para property overviews y portfolio scanning donde la diferenciación visual importa.

Parte 6: Referentes de la Industria

Tabla sintética de cómo major companies abordan agent-native UX:

Compañía	Approach	Innovaciones UX clave	Relevancia para BaW OS
Microsoft Copilot / Copilot Studio	Natural language + structured orchestration	Adaptive Cards, background agent mechanism, intermediate step visualization, Agent Design Canvas framework (Microsoft Design)	Background agents con user-facing mechanism para ver/controlar
Salesforce Agentforce	Modular "primitive components"; estilo probabilístico	Atlas Reasoning Engine, Trust Calibration, Connections (structured UI en 4M sessions), Workspace/Radar pattern (Salesforce Engineering)	UI estructurada (seat maps, radio cards) dentro de respuestas de agente
ServiceNow AI Agents	"AI Experience" — UI para enterprise AI	AI Agent Studio, AI Agent Fabric, AI Control Tower, A2A Protocol, MCP (ServiceNow)	Central AI Control Tower para monitoring de agentes
Palantir AIP	Ontology-first; digital twin	AIP Logic, AIP Evals, Machinery process mining, Data-Logic-Action framework (Zouhall)	Objetos ontológicos (building, unit, contract); HITL gated con audit trail completo
Datadog	Observability extendida a agentes	LLM Observability graph view, Bits AI Agents (SRE, Dev, Security), framework-agnostic tracing, memory state inspection (Datadog)	Modelo mental para observabilidad de agentes BaW
GitHub Copilot (agent mode)	Autonomous real-time collaborator	Reasoning visible en tiempo real, tool access controls, custom instructions, multi-agent sessions (GitHub Blog)	"In the driver's seat" con step-by-step o autonomous switch
Cognition / Devin	Teammate, no tool	Unified interface across planning-to-code-review; conversational + structured (Cognition)	Comunicar con agentes por @mentions
Temporal	Durable workflow orchestration	Web UI con Event History completo, Signals/Queries, durable wait (Temporal)	Base para HITL que resume días después
n8n / LangGraph	Visual canvas + interrupt pattern	Nodos agentes, HITL como first-class node, persistence checkpoints (LangChain Blog)	Automatization builder visible
Guesty	Ops-first property management	Calendar con color-code, widgets real-time, unified inbox con auto-tags (Guesty Blog)	Reservation widgets, Quality Dashboard, Owner Portal
Breezeway	Property care operations	Readiness dashboard con GPS map, task timing, 16+ PMS integrations, mobile app con checklists 10 idiomas (Breezeway)	Single dashboard: cleans + arrivals + maintenance
Linear	Design excellence	Inverted L-shape global chrome, LCH color space, Inter Display + Inter, dark/light first-class (Linear)	Density + hierarchy simultánea

Compañía	Approach	Innovaciones UX clave	Relevancia para BaW OS
Stripe	Financial operations clarity	Table-centric, exception-first notifications, Sigma SQL env, workflow automation (Stripe Docs)	Notificar exceptions (disputes) en home, no vanity metrics

Parte 7: Voces de la Industria y Frameworks Publicados

7.1 Jakob Nielsen — el shift operator-to-supervisor

Contribución más significativa: IA cambia el rol de usuario de **operador** (click cada paso) a **supervisor** (delegar outcomes), forzando UX a evolucionar en 5 dimensiones ([UX Tigers](#)).
Artículos clave:

- "AI Is First New UI Paradigm in 60 Years" — el cursor/mouse/window fue el último paradigma shift
- "Redesigning Workflows for AI" — startups que rediseñaron *workflows end-to-end* en torno a IA generaron 90% más revenue que las que usaron IA para acelerar tareas individuales

7.2 Microsoft Design Team — Agent UX Principles

El framework corporativo más comprehensivo ([Microsoft Design](#)) estructurado en tres dimensiones:

Agent (Space): *Connecting, not collapsing; easily accessible yet largely invisible.* Actúa en background; dashboards visibles para ver/controlar.

Agent (Time): Past (memoria informa tarea actual); Now ("nudging more than notifying"); Future (adapta a devices, modalidades, comportamiento).

Agent (Core): *Embrace uncertainty but establish trust;* transparencia, control y consistencia son non-negotiable; status visible en todo momento.

7.3 Smashing Magazine / Victor Yocco

Los seis patrones core (ver Parte 2.1). Insight central: *"Autonomy is an output of a technical system. Trustworthiness is an output of a design process."* ([Smashing Magazine](#))

7.4 EY Studio — de personas a individuos

"How agentic AI enables a new approach to user experience design" (EY). Introduce la transición de diseñar para **personas** a diseñar para **individuals**, habilitado por real-time context modeling. Nuevos roles emergentes: *intent architect, experience strategist, agent personality designer*. Datos: 64% de organizaciones invirtió \$10M+ en IA en 2024; 67% planea en 2025.

7.5 Salesforce Architect — Agentforce Architecture Pattern Library

Librería enterprise-grade con cuatro archetipos de orquestación (SOMA, MOMA, Multi-Vendor A2A) y ocho patrones de interacción (Greeter, Operator, Orchestrator, Answerbot, Interrogator, Listener/Feed, Workspace/Radar) ([Salesforce Architect](#)).

7.6 ADEPTS (académico)

Ver Parte 2.3. Framework académico que bridgea UX, ingeniería y política ([arXiv 2507.15885](#)).

7.7 Microsoft Research — 12 communication challenges

Identifica 12 challenges organizados en ([Microsoft Research](#)):

- **User-to-Agent (U1–U3):** expresar goals sin ambigüedad; comunicar preferencias/constraints; feedback efectivo
- **Agent-to-User (A1–A5):** comunicar capacidades y límites; mostrar acciones planeadas; execution status; surface side effects; confirmar goal completion
- **General (X1–X4):** ayudar a verificar comportamiento; comportamiento consistente; niveles de detalle apropiados; context management

Hallazgo clave: guidelines HCI previas (Amershi2019, Google PAIR) son **insuficientes** para sistemas agentic — los agentes tienen capacidades generalistas, operan en entornos open-world, e interactúan vía arquitecturas multi-agente complejas.

7.8 VC perspectives: a16z, Sequoia, YC, Deloitte

a16z ([Big Ideas 2026](#), [Developer Patterns](#)): *"AI is moving from chat to action. AI stops being something you ask, and becomes something that does."* Dual-mode interfaces (human + agent facing); MCP como universal standard; accessibility APIs para agentes.

Sequoia ([AI 50 2025](#)): *"AI graduated from an answer engine to an action engine in the workplace."* Vertical agents con RL sobre data sintética + usuario.

Y Combinator ([RFS](#)): AI-Native Enterprise Software ("Cursor for sales, HR, accounting"); infrastructure for multi-agent systems; first 10-person \$100B company.

Deloitte: token-based cost monitoring como requisito nuevo ([Deloitte](#)). "Traditional TCO models need a refresh."

McKinsey ([Agentic Organization](#)): equipos de 2–5 humanos supervisan agent factories de 50–100 agentes; nuevos roles M-shaped (supervisores generalistas) y T-shaped (especialistas para excepciones).

Parte 8: Ingeniería Cognitiva y Diseño Human-AI

8.1 Matriz de complementariedad humano-AI

Del landmark paper PNAS Nexus 2026 ([PNAS Nexus](#)):

Dimensión	Fortaleza IA	Fortaleza humana	División óptima
Filtering/triage	Monitorea a escala; detecta patrones	Detecta edge cases; interpreta contexto	IA hace vigilancia rutinaria; humanos en excepciones
Workload orchestration	Coordina timing, secuencia, escalamiento	Repriorización dinámica, improvisación	IA propone; humanos confirman, adaptan
Reasoning	Procesa data vasta, reglas consistentes	Creatividad, juicio moral, sabiduría contextual	IA genera opciones; humanos pesan valores
Memory	Persistente, completa	Asociativa, contextual, creativa	IA mantiene records; humanos interpretan patrones

El **complementarity gap**: el desempeño conjunto actualmente sigue siendo inferior a IA sola en la mayoría de casos. Los humanos a menudo no muestran **reliance apropiado** ([Taylor & Francis](#)). El reto no es solo construir oversight, sino diseñar *cuándo* se solicita juicio humano y *cómo* se integra.

8.2 Situational awareness (modelo Endsley de 3 niveles)

Framework foundational ([SINTEF](#)):

1. Level 1 — Perception: percibir estado actual de agentes, tareas, entorno
2. Level 2 — Comprehension: entender qué significa esa información para goals actuales
3. Level 3 — Projection: predecir cómo se verán agentes y outcomes en el futuro cercano

El **out-of-the-loop problem**: cuando humanos monitorean sistemas automatizados, son frecuentemente lentos en detectar que se necesita intervención. Una vez detectada, necesitan tiempo adicional para entender el estado lo suficiente para actuar. Este delay causa fallas en cascada.

ATSA (Agent Teaming SA) ([arXiv ATSA](#)): ambos, humanos e IA, deben mantener SA del otro y del entorno compartido. Shared SA: perspectivas compatibles; Distributed SA: diferentes piezas sintetizadas.

8.3 Decision support vs. decision automation

Decision support (IA como tool): cuando expertise humano supera IA; dimensiones éticas centrales; accountability debe quedar con humano. Riesgo: **automation bias**.

Decision automation (IA decide): cuando volumen es demasiado alto, criterios bien definidos, IA verificada, errores reversibles. Riesgo: skill degradation, mode confusion, pérdida de SA.

Hybrid / adaptive: sistema ajusta involvement según contexto, risk level y estado cognitivo humano (Oregon State).

Counter-design clave: hacer que revisar output se sienta **meaningful, no performative**. Binary approve/reject lleva a rubber-stamping; evidence packs + framing contextual mantienen engagement genuino.

8.4 Alert fatigue y notification design

Causas primarias (IBM):

- Volumen overwhelming con contexto insuficiente
- Alta tasa de falsos positivos
- Fragmentación de tools
- Lack de correlated context

Pipeline IA-powered:

1. **Context gathering**: pull telemetría relacionada; excluir unrelated
2. **Correlation**: agrupar alertas del mismo root cause
3. **Enrichment**: asset importance, performance guarantees, risk profiles
4. **Hypothesis generation**: proponer root cause antes de surface al humano
5. **Validation step**: generar steps para que humano confirme
6. **Runbook/automation**: si validado, generar remediation; opcional con HITL gate

Outcomes medidos (Gelman et al. 2023): reducción de response time a incidents accionables en 22.9%; supresión de 54% de falsos positivos (95.1% detection rate); reducción de 14% en alertas por incident.

Principios notification UX: batching, priority tiers, context-in-notification, progressive disclosure, acknowledge protocols, monitoring de fatigue signals (time-to-respond creciente, false-positive dismissal rate, engagement decreciente).

8.5 El out-of-the-loop problem

Cuando humanos monitorean pasivamente por períodos largos, pierden SA. Tesla FSD identifica la solución: **no transferir control instantáneo**; permitir período de assisted co-control durante transición; monitorear tunneling cognitivo antes de transferencia (Stanford FSD, Aptiv whitepaper).

Parte 9: Principios de Diseño Visual para Operaciones

9.1 Sistema de color (RAG + semántico)

Color	Significado	Threshold	Acción
Rojo	Crítico, atención inmediata	>30 días overdue, >10% overage	Escalar a management
Ámbar	Preocupación moderada	Delay dentro de tolerance	Corrective action de equipo
Verde	On track	On budget/schedule	Mantener
Azul	Completado (ext opcional)	Deliverables met	Archivar/celebrar
Gris	No iniciado	Pre-activity	Planning

Accesibilidad (Waydev, Mastt): red-green blindness es la forma más prevalente de CVD; nunca depender solo de color — pair con iconos, text labels, patterns o shapes. Test con CVD filters.

Principio NASA: off-nominal = rojo; within range = **neutro** (no verde activo). Color gestiona atención hacia lo off-nominal, no decora lo normal.

9.2 Typography para high-density

Tres propiedades críticas (FontAlternatives):

Propiedad	Por qué	Implementación
Tabular numerals	Columnas alineadas	font-variant-numeric: tabular-nums
Glyph differentiation	1/1/I y 0/O distinguibles a 11px	IBM Plex Sans, Inter
Compact proportions	Dashboard rara vez >14px; x-height alto pack más	Inter, Source Sans 3

Recomendaciones de fuente:

Fuente	Mejor para	Ventaja
Source Sans 3	Analytics dashboards (top pick)	Mejor hinting cross-platform; optical size axis; variable
Inter	Enterprise dashboards	x-height alta, clara a 11px, glyphs distintivos
IBM Plex Sans	Institutional/enterprise	Diseñada para data accuracy interfaces
Barlow	Layouts constrained	Condensed variants
Noto Sans	Multilingual	Free con cobertura multi-script

Settings: font-size: 13px + line-height: 1.4 para celdas de tabla; tabular-nums globalmente en dashboard containers. No usar monospace para data tables — tabular en proportional da columnas alineadas con mejor readability.

9.3 Dark mode best practices

Cuándo vale la pena: apps donde usuarios pasan horas en pantalla (monitoring, NOCs, 24/7), pop de status indicators, ambientes low-light.

Best practices:

1. Evitar negro puro (#000000) — usar grays oscuros (#121212, #1A1A1A)
2. Evitar saturados — desaturar ~20 puntos vs light mode (optical vibrations)
3. Comunicar elevación con surfaces más claras — en vez de drop shadows
4. Card layering: #FFF @ 8–10% opacity + inset white border @ 12–15%
5. No solo invertir colores — palette separada, testing

Approach Linear (Linear): LCH color space para theme generation; different elevations con different LCH; text/icons más dark en light mode, más light en dark para contrast consistente.

9.4 Badge & tag systems

Taxonomía clara (Cieden, LinkedIn/Friedman):

Componente	Tipo	Uso	Interactive?
Badge	Static	Status updates (draft, new, pending, -7%↘)	No
Status indicator	Static	Real-time statuses (online, live, blocked)	No
Static tag	Static	Topics, keywords (AI, Accesibilidad)	No
Filter tag / chip	Interactive	Filters, selection, removable	Yes

Reglas: labels estáticos no deben parecer botones; posicionar cerca de headers o items que describen; badge text 1–3 palabras max.

Patrón para operaciones: IN PROGRESS, BLOCKED, OVERDUE, AT RISK, COMPLETE, NEEDS REVIEW. Pair color + text + icon opcional. Time-since indicators: "3h" o "2d" para context.

9.5 Densidad: visual, información, temporal

Ver Parte 5.2. Las tres trabajan juntas: reducir visual density permite más información density, que con temporal density alta (speed) produce value density alta.

Parte 10: Recomendaciones para BaW OS

10.1 Principios fundamentales de diseño

Derivados de la síntesis completa, 14 principios gobiernan BaW OS:

1. Humans supervise, agents operate. La UI default está optimizada para supervisión; ejecución ocurre en background, surfaceando solo lo que requiere juicio humano.

2. **Separar intent de execution.** Todo agente *propone* antes de *commit-ear*. Ninguna acción consecuente se ejecuta sin estar en uno de tres tiers: AUTO, LOG, REQUIRE_APPROVAL ([Stack AI](#)).
3. **Exception-first design.** El default es "el sistema corre y no requiere tu atención". Las excepciones surfacean; las operaciones rutinarias quedan invisibles ([ProSense](#)).
4. **Autonomy earns, never assumes.** Todo agente inicia en A0–A1 (Advisory / Assistive). Sube a A2/A3 con data: >95% accuracy over hundreds of reviewed interactions ([Innovate 24-7](#)).
5. **Trust is calibrated, not declared.** Mostrar confidence, rationale, evidence y undo en todas las acciones consecuentes. Binary signals ("confío" / "no estoy seguro") funcionan mejor que porcentajes ([FuseLab](#)).
6. **Transparency, control, consistency son non-negotiable.** Status del agente visible en todo momento; controles accesibles en todo momento; comportamiento predecible ([Microsoft Design](#)).
7. **Everything is auditable.** Cada acción de agente + intervención humana es un first-class event con attribution metadata mínima. Audit log es surface de UI, no debug tool ([MarkTechPost](#)).
8. **Task-centric, no entity-centric.** Las pantallas se organizan por *qué necesita atención y qué acción tomar*, no por tablas DB ([GlitchLabs](#)).
9. **Ontology over tables.** El edificio, la unidad, el contrato, el cobro son objetos ontológicos componibles en vistas por rol (á la Palantir) ([Zouhall](#)).
10. **Least-privilege delegated identity.** Cada agente tiene su propia identidad con scoped, just-in-time, revocable credentials ([Oso](#)).
11. **Plan-level oversight, not step-level rubber-stamping.** Aprobar el plan es más efectivo que aprobar cada paso ([arXiv oversight](#)).
12. **Progressive disclosure, not information dump.** Summary por default; expand on demand para Why; nivel profundo solo para auditores/power users.
13. **Dual-channel by design.** Cada surface sirve a humanos y a agentes. Semantic APIs + machine-readable interfaces paralelos a la UI humana ([a16z](#)).
14. **Feedback loop from day one.** Cada rechazo, cada override, cada escalation feed back a mejorar al agente. *"The part most teams build last — and why the same escalations recur indefinitely."*

10.2 Arquitectura de información

Jerarquía ontológica:

```
Organization (developer / property management company)
├── Portfolio / Building (edificio individual)
│   ├── Unit (unidad habitacional / comercial)
│   │   ├── Contract (lease, booking, ownership)
│   │   └── Reservation (si aplica)
```

- └─ Maintenance item
- └─ Financial transactions

Modelo de navegación recomendado (síntesis Linear + Stripe + BMS):

- Inverted L-shape como chrome global (sidebar + top bar)
- Sidebar primario (240–300px expanded, 48–64px collapsed): organización/portfolio selector + navegación por rol
- Top app header: search global, command palette (⌘K), agent status indicator, user
- View headers: filtros + display options + switcher de vistas (tabla/card/mapa/calendar/board)
- Side panel / bottom panel para detail (side cuando ≤5 columnas importantes)

Vistas por rol (role-based progressive disclosure):

Rol	Vista primaria	Énfasis	Agentes visibles
Developer / Portfolio Owner	Executive Mission Control cross-portfolio	KPIs financieros + ocupación + exceptions	Agent ROI, fleet status
Ops Manager	Operations Dashboard	Readiness de hoy + tareas + excepciones + map	Full agent operations layer
Concierge / Front Desk	Unified Inbox + today's arrivals	Comms + check-ins + issues	Guest comms agent acciones
Property Owner	Owner Portal	Revenue + ocupación + maintenance + statements	Transparencia en agente autónomas
Tenant / Guest	Tenant portal	Su contrato, su cobro, su comms, su maintenance	"Your AI concierge handled X"
AI Agent (dual channel)	API / MCP surface	Structured state, intent endpoints	—

10.3 Patrones de interacción agent-native

La autonomy dial para BaW OS (per workflow, per building):

Nivel	Descripción BaW	Ejemplos de workflows
L1— Observe & Suggest	Agente detecta, no actúa	Flagging de anomalies en consumo de servicios
L2 — Plan & Propose	Agente crea plan, humano revisa cada uno	Dunning sequence para inquilinos con atraso
L3 — Act with Confirmation	Agente prepara, humano da go/no-go	Emisión de facturas, respuestas a huéspedes
L4 — Act Autonomously	Agente ejecuta, notifica después	Confirmación de reservas, smart-lock access, standard maintenance scheduling

Workflows de aprobación por dominio:

Dominio	AUTO	LOG	REQUIRE_APPROVAL
Collections	Envío de recordatorios standard	Envío de warnings de atraso 15-30d	Corte de servicio, acción legal, write-off
Maintenance	Scheduling rutinario, asignación a staff	Costo <threshold, emergency menor	Costo >\$X, vendor nuevo, afecta ocupación
Guest comms	FAQs, confirmaciones	Respuestas con tono custom	Disputes, refunds, reviews públicas response
Pricing / Revenue	Ajustes <5% según market data	Ajustes 5-15%	Ajustes >15%, cambios de política
Leasing	Pre-screening de aplicantes	Responses rutinarias	Aprobación final de lease, concessions
Financial	Reconciliaciones	Emisión de receipts	Transferencias, pagos a vendors, refunds

Visibilidad del agente en building operations:

- Agent status bar siempre visible en top (idle/running/paused con contadores)
- Agent attribution en cada dato tocado por agente: icono distintivo + tooltip con rationale
- Agent activity feed como nav destination de primera clase
- Fleet view para owners con múltiples buildings: qué agentes corren, missions activas, human oversight needs

Flujos de escalamiento específicos (tipos de escalation ajustados a property ops):

1. Clarification — Agente necesita info faltante: "La dirección del nuevo owner está incompleta; necesito el RFC para emitir el statement"
2. Decision — Múltiples paths válidos: "Tres ofertas de vendor plomería para U-204; ¿priorizamos precio, tiempo o rating?"
3. Validation — Riesgo alto requiere sign-off: "Propongo waive 50% late fee a Sr. García; 1er atraso en 18 meses"
4. Context — Situación novel: "Inquilino solicita extender lease 3 meses; no hay política previa para este unit type"
5. Error — Falla técnica: "API del banco timeout en cobro automático; reintentar, manual, o flagged?"

10.4 Pantallas clave y recomendaciones de diseño

Executive Mission Control

- Patrón: Exception-first dashboard + KPI summary + trend sparklines
- Data a surfacear: NOI por edificio (RAG), ocupación vs benchmark, critical alerts (top), agent ROI (missions completed vs human-equivalent hours), cash position
- Agentes visibles: "Hoy tus agentes completaron X missions; 3 requieren tu atención"

- **Layout:** Top section = exceptions; middle = trend charts (ocupación, revenue, costos); bottom = portfolio summary + drill-down links

Operations Dashboard

- **Patrón:** Breezeway-style readiness dashboard + GPS map + split view
- **Data:** Status de todas las unidades hoy (clean/dirty/inspecting/ready), check-ins/check-outs, arrivals, maintenance tasks, staff en campo (GPS), SLA countdowns
- **Agentes:** Agent operations panel lateral con missions activas + needs attention
- **Layout:** Primary = table de unidades con color-coded status; detail = unit details + task timeline; map toggle; filtros por building/floor/status

Units (Gestión de unidades)

- **Patrón:** Table-centric con split view (side panel para ≤ 5 cols, bottom para > 5)
- **Data:** Unit ID, status, contract actual, tenant, rent, last maintenance, occupancy rate, next event
- **Agentes:** Columna "Agent activity" con latest autonomous action + confidence
- **Layout:** Filtros por building/floor/status; vista card como toggle para portfolio scan

Contracts

- **Patrón:** Lista + detail page + state-machine visual
- **Data:** Tenant/lessor, unit, término, rent, status (draft/active/renewing/ending/terminated), last interaction
- **Agentes:** Renewal suggestions, dunning history, document drafts pendientes
- **Layout:** Tabs por estado (Active, Renewal pending, Ending soon, Terminated); detail con timeline de eventos

Collections

- **Patrón:** Exception-first queue + evidence pack per item
- **Data:** Atrasos por bucket (0–15d, 16–30d, 31+d), promesas de pago, gestiones agentic en curso, acciones pendientes de aprobación humana
- **Agentes:** Full dunning agent visibility con autonomy level per tenant/risk
- **Layout:** Top = aging summary + exceptions; middle = action queue agentic; bottom = historical recoveries

Maintenance

- Patrón: Kanban + mapa + task queue híbrido
- Data: Tickets por estado, SLA countdowns, asignado (staff / vendor / agent), costo estimado, unit impact
- Agentes: Auto-triaged tickets, vendor suggestions, scheduling proposals
- Layout: Kanban columns (New / Assigned / In Progress / Awaiting Parts / Done); toggle a mapa; filter por building/priority

Owner Portal

- Patrón: Stripe-style financial clarity + progressive disclosure
- Data: Revenue YTD, ocupación vs benchmark, pending statements, maintenance approvals pendientes, agent activity summary
- Agentes: "Tu AI property manager hizo X esta semana" con link a audit log
- Layout: Top hero con KPIs; sections por categoría; drill-down a statements y transactions

Tenant Portal

- Patrón: Mobile-first minimal + unified inbox
- Data: Contrato actual, rent status, maintenance tickets, comms, documents
- Agentes: AI concierge chat embedded; transparent re: qué el agente puede hacer autonomously vs requires staff
- Layout: Single-page mobile; tabs para sections

Agent Operations Layer

- Patrón: Mission control para la flota de agentes
- Data: Cada agente (name, role, autonomy level, status idle/running/paused), current missions, KPIs (success rate, human intervention freq, cost), recent activity
- Layout: Overview panel arriba (on/off toggles, metrics estilo Tesla); mission list central; drill-down a mission detail con task-by-task breakdown
- Hardware-inspired toggle con confirmación blocking: "¿Seguro quieres apagar este agente?"

Audit Log

- Patrón: Unified action feed con filtros por actor
- Data: Timeline cronológico de acciones agentic + humanas, diff view, confidence trace, rollback hooks
- Layout: Timeline vertical densa con color-coding [AGENT] (verde/azul) [HUMAN] (naranja) [SYSTEM] (gris); búsqueda + filtros por actor/entity/action type; export para compliance

Automations

- Patrón: n8n-style visual canvas + natural language trigger builder
- Data: Triggers configurados, missions definidas, performance de cada automatization
- Layout: Library de templates; canvas para custom; natural-language trigger textbox; backtesting ("estas conditions pasaron 23 veces la semana pasada")

10.5 Sistema visual

Palette recomendada (LCH-generated à la Linear):

- Base: greys neutros #0E1116 (dark), #F8F9FB (light) con jerarquía de superficies en 4 niveles
- Accent: azul distintivo moderado — usado con reserva, solo en chrome y estados default
- Status (RAG semántico):
 - Critical/Red: #E5484D (desaturado para dark)
 - Warning/Amber: #F1A10D
 - Success/Green: #30A46C
 - Info/Blue: #3E63DD
 - Neutral: greys scale
- Agent-specific: un tono distintivo (sugerido violeta/cyan) que identifique acciones autonomas del agente — usar con consistencia para que sea escaneable

Typography:

- Primary: Inter (UI) con font-variant-numeric: tabular-nums global
- Headings: Inter Display (más expresivo, mismo xx-height)
- Numerics: Source Sans 3 como alternativa para tablas densas (mejor hinting a <12px)
- Table cells: 13px / line-height 1.4 / tabular-nums

Status color system (pairing color + icon + text):

- CRITICAL rojo + ✕ + text
- AT RISK amber + ⚠ + text
- IN PROGRESS blue + 🔄 + text
- COMPLETE green + ✓ + text
- NEEDS REVIEW violeta + ○ + text (agent-specific)
- PAUSED grey + ■■ + text
- UNDONE strikethrough + ✖ + text

Component patterns:

- Badges: 3 niveles de emphasis (solid, soft, outline), 1-3 words

- Action receipts con qué cambió + diff + undo window
- Confidence signals como chip discreto en inputs agentic
- Role cards para agent identity (avatar + name + role + autonomy level + status)
- Evidence panels colapsables con citations y rationale
- Empty states celebrando "Inbox Zero" como estado operativo ideal

Dark vs. light mode recommendation: Ambas first-class. Dark default para ops teams (hours-long use, exception pop, low-light environments typical de NOCs / late-shift concierge); light default para owner portal y tenant portal (casual use, readability). Toggle en user settings + system preference respect. LCH color space para consistency (Linear).

10.6 Las 3 direcciones de concepto

Para exploración de diseño, tres direcciones conceptuales:

Direction A: Executive Minimal

Brief: Minimalism ejecutivo tipo Notion / Linear 2025. Mucho whitespace, tipografía grande, jerarquía clara, color usado con extrema reserva. Density baja-media.

Pros: Claridad superior para owners y executives. Onboarding rápido. Visualmente premium.

Contras: Insuficiente para power users ops que necesitan ver muchas unidades/tareas a la vez. Requiere muchos clicks para operaciones densas.

Cuándo elegir: Si el usuario primario es developer/owner/C-level, no ops team.

Direction B: Dense Operations

Brief: Bloomberg Terminal-adjacent. Tablas densas, muchos KPIs visibles simultáneamente, keyboard-driven, workspaces componibles, dark mode default. Alta visual + information density.

Pros: Poder superior para ops teams trained. Temporal density alta. Escalable a portfolios grandes.

Contras: Curva de aprendizaje empinada. Intimidante para owners, tenants, y staff nuevo. Onboarding costoso.

Cuándo elegir: Si el producto se vende a property management companies grandes con ops teams profesionales full-time.

Direction C: Hybrid Agent-Native Command Center ★ RECOMENDADA

Brief: Híbrido que toma lo mejor de ambas. Dense por default para ops views (Operations Dashboard, Units, Maintenance, Audit); minimal para portals (Owner, Tenant) y ejecutivo (Mission Control). **Agent-native:** status del agente siempre visible, agent inbox como primer citizen, exception-first en todas las vistas, autonomy dial por workflow, action receipts con undo.

Estética: Linear-style LCH palettes, dark mode default para ops, light default para portals, Inter + Source Sans 3, componentes modulares. Violeta/cyan distinctive para agent actions.

Estructura: Inverted L-shape chrome (sidebar + top bar), split views como workhorse, command palette `⌘K` para power users, progressive disclosure de 3 niveles, table-centric para ops + card-centric para portfolio scan toggleable.

Pros: Sirve todos los roles sin comprometer density donde importa. Agent-native de origen. Escalable desde 1 edificio a portfolios masivos.

Contras: Mayor complejidad de diseño y desarrollo. Requiere design system riguroso.

Por qué se recomienda: La investigación converge fuertemente en que el futuro es híbrido — Linear's density-con-hierarchy, Microsoft's agent-native principles, Breezeway's operations focus, Stripe's financial clarity. Comprometer en cualquiera de esas dimensiones deja valor sobre la mesa.

10.7 Checklist de diseño agent-native para BaW OS

Antes de diseñar:

- ☐ Mapear workflow a nivel de **decisión**: qué requiere juicio humano, qué puede ser agentic
- ☐ Definir action taxonomy per dominio (AUTO / LOG / REQUIRE_APPROVAL)
- ☐ Definir escalation rules: thresholds, paths, authorities
- ☐ Definir agent identity model: permisos, scoping, temporalidad, audit
- ☐ Elegir autonomy starting point (conservador: A0–A1 por default)

Componentes UX core:

- ☐ Intent Preview (Plan Summary) con `[Proceed]` `[Edit]` `[Handle Myself]` para acciones significativas/irreversibles
- ☐ Autonomy Dial per workflow/risk tier (Observe → Suggest → Confirm → Autonomous)
- ☐ Explainable Rationale ("Because X, I did Y") para toda acción autonomous
- ☐ Confidence Signal visual (binary + optional numeric)
- ☐ Action Audit Log unificado (humano + agente), filtrable por actor
- ☐ Undo/Rollback con ventana clara y comunicación honesta cuando no es reversible
- ☐ Escalation Pathway con 5 tipos (clarification / decision / validation / context / error)
- ☐ Start/Stop/Pause/Resume controls para long-running tasks
- ☐ Evidence Panel con citations challengeable
- ☐ Human Checkpoint Gates tipados (plan / execution / output / exception)
- ☐ Memory Controls visibles (qué recuerda, por qué, editable)
- ☐ Budget + Time Boxes con real-time ETA/cost

Estados agentic diseñados:

- ☐ Suggested by Agent (pending review)
- ☐ Pending Approval (con SLA countdown)
- ☐ Approved
- ☐ Executing (progress, current step, ETA)
- ☐ Completed (con action receipt)
- ☐ Executed by Agent (con undo window)
- ☐ Escalated to Human (full context)
- ☐ Blocked (resolution path)
- ☐ Failed (recovery options)
- ☐ Undone (before/after)

Pantallas críticas:

- ☐ Executive Mission Control
- ☐ Operations Dashboard
- ☐ Units / Contracts / Collections / Maintenance
- ☐ Owner Portal + Tenant Portal
- ☐ Agent Operations Layer (fleet management)
- ☐ Audit Log (unified, compliance-ready)
- ☐ Automations builder (visual + natural language)
- ☐ Empty states de "Inbox Zero" como first-class

Sistema visual y accesibilidad:

- ☐ LCH-based color system con RAG semántico
- ☐ Typography con tabular-nums global
- ☐ Dark + light mode first-class
- ☐ Badge/status system consistente
- ☐ CVD testing (red-green blindness)
- ☐ Keyboard shortcuts + command palette
- ☐ Mobile-first para portals y field ops

Compliance y governance:

- ☐ Audit logs architecturally enforced (no procedural)
- ☐ Agent identity separada de humans
- ☐ Scoped + time-limited credentials per task
- ☐ Human interventions as first-class events
- ☐ Attribution metadata mínima por acción (agent ID, model, prompt hash, input hash, confidence, human review status)

- [] Ready para EU AI Act Article 12(1)

Lo que NO hacer:

- [] No mostrar chat como único progress indicator
- [] No mostrar "Done!" sin action receipt
- [] No pedir aprobación para todo (rubber-stamping mata el purpose)
- [] No usar service account compartido entre humano y agente
- [] No tratar escalation como error state
- [] No loguear a wall-of-text; estructurar
- [] No construir dashboard estático; hacerlo exception-first

Appendix: Lista de Fuentes

Agent-Native UX y Patrones

- ReloadUX — [Agent-native workflows when AI replaces UI](#)
- Smashing Magazine — [Designing For Agentic AI: Practical UX Patterns For Control](#)
- HatchWorks — [Agent UX Patterns](#)
- Onething Design — [Agentic AI UX Design](#)
- TianPan — [Designing Approval Gates for Autonomous AI Agents](#)
- Stack AI — [HITL AI Agents: Designing Approval Workflows](#)
- MarkTechPost — [Transparent AI Agents with Audit Trails](#)
- UXmatters — [Designing for Autonomy: UX Principles for Agentic AI](#)
- Aufait UX — [Agentic AI Design Patterns Enterprise Guide](#)
- Bean Machine — [Agent-First Design Paradigms](#)
- The Interactive Studio — [Beyond the Chat: Agentic Interfaces](#)
- Benjamin Prigent — [7 UX patterns for human oversight in ambient AI agents](#)
- SaaS Factor — [Traditional SaaS vs Agentic AI UX](#)
- Fuselab Creative — [UI Design for AI Agents](#)
- LukeW — [Agent Management Interface Patterns](#)

Voices, Frameworks & Thought Leadership

- UX Tigers / Jakob Nielsen — [Intent by Discovery](#)
- Microsoft Design — [UX Design for Agents](#)
- Microsoft Design — [The New UI for Enterprise AI](#)
- Microsoft Learn — [Copilot Studio Agent Design Canvas Framework](#)

- Salesforce Architect — [Enterprise Agentic Architecture](#)
- Salesforce Architect — [The Agentic Enterprise IT Architecture](#)
- Salesforce Engineering — [How Agentforce Converts LLM Responses to Structured UI](#)
- EY Studio — [How agentic AI enables a new approach to UX design](#)
- ADEPTS Framework — [arXiv:2507.15885](#)
- Microsoft Research — [Human-Agent Communication Challenges](#)

VC and Industry Analysis

- a16z — [Nine Emerging Developer Patterns](#)
- a16z — [Notes on AI Apps in 2026](#)
- a16z — [Big Ideas 2026](#)
- Sequoia — [AI 50 2025](#)
- Y Combinator — [Requests for Startups](#)
- McKinsey — [The Agentic Organization](#)
- Deloitte — [AI Tokens: Spend Dynamics](#)
- Deloitte — [AI and the future of human decision-making](#)
- Glean — [Will AI Agents Replace SaaS Tools](#)
- AI Space — [The Death of SaaS](#)
- LBZ Advisory — [The Death of the Dashboard](#)
- Oliv — [The Death of SaaS Dashboards](#)
- Press Ganey — [The future of agentic AI](#)
- Curious Compass — [Ambient AI Enterprise](#)
- Flashdba — [AI Can Replace Interfaces, Not Systems of Record](#)
- Mercury Media Technology — [SoR in Age of AI](#)

Enterprise Products

- Microsoft VS Code Blog — [Multi-Agent Development](#)
- Microsoft Learn — [Request information from humans in agent flows](#)
- ServiceNow — [AI Agents](#)
- Palantir Zouhall Analysis — [Palantir AIP Deep Dive](#)
- Palantir — [AIP Agent Studio](#)
- Datadog — [Monitor AI Agents](#)
- Datadog — [Datadog AI Innovation](#)
- Datadog — [Executive Dashboards](#)

- GitHub Blog — [Agent Mode 101](#)
- Cognition — [Introducing Devin 2.2](#)
- Temporal — [Orchestrating Ambient Agents](#)
- Temporal Learn — [Human-in-the-Loop Tutorial](#)
- LangChain Blog — [Making HITL easier with interrupt](#)
- n8n — [AI Agentic Workflows](#)
- Relevance AI — [Changelog](#)
- UiPath — [Human-Robot Collaboration](#)
- Dust — [AI Agents Examples](#)
- Oracle Fusion COE — [Human Approval Node](#)

Command Center, Operations, and Design

- UX Planet — [Mission Control Software UX Design Patterns](#)
- NASA — [Cockpit Display Design](#)
- ExterNetworks — [NOC Design and Layout](#)
- CCS Projects — [Designing Command and Control Centers](#)
- Cerexio — [HMI in SCADA Guide](#)
- Simio — [Digital Twin Dashboard Design](#)
- Linear — [How we redesigned the Linear UI](#)
- Linear — [Linear Liquid Glass](#)
- Stripe — [Dashboard Basics](#)
- PagerDuty — [Incident Workflows Deep Dive](#)
- Palantir — [Platform Overview](#)
- Guesty — [Quality Dashboards](#)
- Breezeway — [Operations Platform](#)
- Hostaway — [Operations](#)
- Tanwui — [BMS UX UI Design](#)
- Bloomberg — [Innovating a Modern Icon](#)
- Matt Ström-Awn — [UI Density](#)
- Lippihom — [High-Information-Density Interfaces](#)
- ProSense — [Exception-Based Dashboard Design](#)
- GlitchLabs — [Admin Dashboard UX Patterns](#)
- Cloudscape — [Split View Pattern](#)
- PatternFly — [Primary-Detail](#)

- Waydev — [RAG Status Scheme](#)
- FontAlternatives — [Best Fonts for Dense Dashboards](#)
- Design Studio UI/UX — [Dark Mode Best Practices](#)
- Cieden — [Badge UI Design](#)

Human-Agent Collaboration

- Knight Columbia — [Levels of Autonomy for AI Agents](#)
- CEI America — [A0-A4 Framework](#)
- LlamaIndex — [Confidence Threshold](#)
- Turing — [Human-Guided AI](#)
- Innovate 24-7 — [Human-on-the-Loop](#)
- Oso — [AI Agent Permissions](#)
- IEEE Standards — [Four Conditions for Trusted AI](#)
- Seekr — [Explainable AI Enterprise Guide](#)
- Seekr — [Citation-Based AI Agents](#)
- Taylor & Francis — [Between Transparency and Trust](#)
- Taylor & Francis — [Complementarity in Human-AI Collaboration](#)
- PNAS Nexus — [Toward a science of human-AI teaming](#)
- Anthropic — [Measuring Agent Autonomy](#)
- arXiv — [Comparing Human Oversight Strategies](#)
- arXiv — [ATSA Framework](#)
- SINTEF — [Human-AI Teaming Research](#)
- IBM — [Alert Fatigue Reduction](#)
- AgileSoftLabs — [Multi-Agent AI Systems Guide](#)
- Azure — [AI Agent Design Patterns](#)
- Arion Research — [Conflict Resolution Playbook](#)
- Frontiers in Robotics — [FORMIGA](#)
- Stanford — [Autonomous Car Control](#)
- Aptiv — [Automated Driving Human-Vehicle Interaction](#)
- Oregon State — [Research on Human-AI Teaming](#)
- Augment Code — [Multi-Agent Outputs Enterprise Audit](#)
- Token Security — [Transparency and Explainability](#)
- Rosenfeld Media — [The Handoff is Dead](#)
- Notion — [Notion 3.0 Agents Release](#)

- Figma — 2025 AI Report
- Shopify Engineering — MCP UI
- Mobisoft Infotech — AI UX 2025: Rise of AX

Reporte compilado de cuatro investigaciones primarias (agent-native UX patterns, command center UI, human-agent collaboration, agentic SaaS trends 2025–2026), con síntesis aplicada al diseño de BaW OS, una plataforma agent-native de operaciones inmobiliarias.